Spider Smart Solution

Inventory Management System

Product Requirements Document — Final v4.0

Phase 1: Complete IMS | All Client Requirements Satisfied

Version	4.0 — Final (All requirements gap-filled)
Status	Final — Ready for Development
Date	April 2026
Prepared By	Spider Smart Solution — Product Team
Stack	React + Vite | FastAPI (Python) | Supabase (PostgreSQL)
AI / Automation	**Deferred to Phase 2 — free/open-source LLMs (Groq, Gemini, Ollama)**
Changes from v3.0	**Added: External Guest role, Tags & taxonomy, Saved searches, PDF export, Version history + revert, Record locking, Retention & disposition module, Legal hold, Custom report builder, eDiscovery, Read-audit logging, Configurable field types**

1. Gap Analysis — v3.0 vs Client Requirements

A full review of the original requirements document was conducted against PRD v3.0. The table below lists every gap found and how it is resolved in this final PRD.

#	Requirement from Client Doc	Status in v3.0	Resolution in v4.0
1	External Guest role	❌ **Missing**	☑ **Added — read-only external access with expiry**
2	Configurable record types & field schemas	❌ **Missing**	☑ **Added — Admin can define custom field types per record type**
3	Taxonomy, tags, hierarchical categories	❌ **Missing**	☑ **Added — tag engine + category hierarchy**
4	Auto-classification rules (rule-based, not AI)	❌ **Missing**	☑ **Added — rule-based auto-tagging on record save**

#	Requirement from Client Doc	Status in v3.0	Resolution in v4.0
5	Saved searches	❌ Missing	☑ Added — users can save & re-run searches
6	Result export to PDF	❌ Missing	☑ Added — PDF export via WeasyPrint on backend
7	Version history with diff & revert	⚠ Partial — audit log only, no revert	☑ Added — full version snapshots, diff view, revert
8	Record locking / optimistic concurrency	❌ Missing	☑ Added — optimistic locking via updated_at ETag
9	Retention compliance reports & upcoming dispositions	❌ Missing	☑ Added — retention module with policy config
10	Legal holds preventing disposition	❌ Missing	☑ Added — legal hold flag blocks disposition
11	Custom report builder	❌ Missing	☑ Added — drag-field report builder in UI
12	Scheduled exports (CSV/PDF)	❌ Missing	☑ Added — cron-based scheduled report emails
13	eDiscovery / chain-of-custody metadata	❌ Missing	☑ Added — legal export package with CoC log
14	Read-action audit logging	❌ Missing (write only)	☑ Added — READ/EXPORT/SEARCH logged optionally
15	Faceted search (record type, retention status)	⚠ Partial — basic filters only	☑ Added — faceted filter panel with all dimensions
16	XLS import (not just CSV)	❌ Missing	☑ Added — openpyxl handles .xlsx import
17	Record type presets / templates	❌ Missing	☑ Added — Admin can create form templates
18	Core 8 required fields + validation	☑ Covered	☑ Unchanged — all 8 fields fully validated
19	Role-based access (4 roles)	☑ Covered	☑ Extended to 5 roles (+ External Guest)
20	Full-text search & advanced filters	☑ Covered	☑ Extended with faceted search & saved searches
21	Immutable audit log with field-level diff	☑ Covered	☑ Extended with read-audit + tamper-evident flag

#	Requirement from Client Doc	Status in v3.0	Resolution in v4.0
22	Bulk CSV import/export	☑ Covered	☑ Extended to include XLS + schema mapping

2. Executive Summary

Spider Smart Solution requires a web-based Inventory Management System (IMS) to create, classify, search, retain, and audit metadata records for physical assets — primarily boxes and files identified by barcodes — across multiple entities, departments, and locations.

This final PRD (v4.0) satisfies all requirements from the client's requirements document. It defines a complete Phase 1 build: a fully functional metadata management platform with structured record capture, configurable field schemas, tag-based classification, retention management, legal holds, full audit compliance, and advanced reporting — with no AI features (deferred to Phase 2 using free LLMs).

The tech stack uses React (frontend), FastAPI/Python (backend), and Supabase PostgreSQL (database) — all developer-friendly, well-documented, and free to run at the scale required for Phase 1.

3. Tech Stack — Final

Layer	Technology	Free?	Purpose
Frontend	React + Vite	☑ Free	All UI — SPA with React Router
UI / Styling	shadcn/ui + Tailwind CSS	☑ Free	Accessible, professional component library
Charts	recharts	☑ Free	Bar, pie, and line charts for reports dashboard
State Management	Zustand	☑ Free	Auth state + global UI state
Form Validation	React Hook Form + Zod	☑ Free	Client-side form validation — mirrors Pydantic rules
HTTP Client	Axios	☑ Free	API calls with auth interceptor
Backend	FastAPI (Python 3.12)	☑ Free	REST API — all business logic, auth, validation
Data Validation	Pydantic v2	☑ Free	Strict input validation on every endpoint
ORM	SQLAlchemy 2.0 (async)	☑ Free	Python ORM — all DB queries

Layer	Technology	Free?	Purpose
Migrations	Alembic	☑ Free	Version-controlled schema migrations
PDF Generation	WeasyPrint (Python)	☑ Free	Server-side PDF export for reports and records
XLS Processing	openpyxl (Python)	☑ Free	Read .xlsx import files; write .xlsx exports
CSV Processing	Python csv module	☑ Free	CSV import/export — zero dependencies
Scheduling	APScheduler (Python)	☑ Free	Scheduled report jobs & retention checks — no Redis needed
Database	Supabase (PostgreSQL 15)	☑ Free tier	Managed PostgreSQL — all records, audit, versions
Auth	FastAPI + JWT (python-jose + bcrypt)	☑ Free	Email + password; JWT tokens; role claims
Email	FastAPI-Mail + Gmail SMTP	☑ Free	User invites, scheduled report delivery
Frontend Hosting	Vercel	☑ Free tier	Git-push deploy; CDN-backed
Backend Hosting	Railway or Render	☑ Free tier	Docker container deploy for FastAPI

4. User Roles & Permissions

Role	Description	Key Permissions
System Admin	Full system control	All CRUD; user management; configure record types, fields, taxonomies, retention policies; full audit access; system settings
Records Manager	Manages records lifecycle	Create/edit/delete all records; apply/remove tags; manage retention; apply legal holds; bulk import/export; all reports; view audit log
Knowledge Worker	Day-to-day data entry	Create records; edit own records; search & view all; apply tags; save searches; no delete; no user management
Auditor	Read-only compliance access	View all records and full audit log; export audit log; run all reports; cannot create or modify anything

Role	Description	Key Permissions
External Guest	NEW — Limited external read access	View only a permitted subset of records (scoped by Admin); no edit, no delete; no audit log; access expires on configured date

5. Data Model

5.1 Core Required Fields (from Client Requirements)

Field	DB Column	Type	Required	Validation Rule
Entity	entity	VARCHAR(100)	Yes	Alphanumeric; from Entities master list (free-text in Phase 1, dropdown in Phase 2)
Entity Code	entity_code	SMALLINT	Yes	Integer between 10 and 99 (exactly 2 digits)
Department	department	VARCHAR(100)	Yes	Alphanumeric; from Department master list
Location	location	VARCHAR(16)	Yes	Max 16 characters
Box Barcode	box_barcode	CHAR(11)	Yes	Exactly 11 characters; globally unique
File Barcode	file_barcode	CHAR(13)	Yes	Exactly 13 characters; globally unique
Description	description	TEXT	Yes	No max length
Year	year	SMALLINT	Yes	4-digit integer (e.g. 2024)

5.2 System & Extended Fields

Field	DB Column	Type	Description
Record ID	id	UUID	Generated on insert
Record Type	record_type_id	UUID (FK)	Links to configurable record_types table
Tags	tags	TEXT[]	Array of tag slugs applied to this record
Category	category_id	UUID (FK)	Hierarchical category from taxonomy tree

Field	DB Column	Type	Description
Version	version	INTEGER	Increments on every update; starts at 1
Retention Policy	retention_policy_id	UUID (FK)	Links to configured retention policy
Retention Due Date	retention_due_date	DATE	Calculated: Year + retention period years
Disposition Status	disposition_status	ENUM	ACTIVE, DUE, DISPOSED, LEGAL_HOLD
Legal Hold	legal_hold	BOOLEAN	TRUE blocks disposition; default FALSE
Legal Hold By	legal_hold_by	UUID (FK)	User who applied the legal hold
Legal Hold At	legal_hold_at	TIMESTAMPTZ	When legal hold was applied
Is Active	is_active	BOOLEAN	FALSE = soft-deleted; default TRUE
Created By	created_by	UUID (FK)	User ID from JWT on insert
Created At	created_at	TIMESTAMPTZ	Set on INSERT
Updated By	updated_by	UUID (FK)	User ID from JWT on UPDATE
Updated At	updated_at	TIMESTAMPTZ	Used as ETag for optimistic locking
Custom Fields	custom_fields	JSONB	Key-value store for configurable extra fields defined in record_types

5.3 All Database Tables

Table	Purpose
inventory_records	Core records with all fields above
record_versions	Snapshot of every version of a record — enables diff and revert
record_types	Configurable record type definitions (name, description, icon)
record_type_fields	Custom field definitions per record type (field name, type, required, default, validation)
users	System users (id, email, hashed_password, role, is_active, expires_at for Guest, last_login)
entities	Entity master list
departments	Departments per entity (entity_id FK)
locations	Physical location master list

Table	Purpose
tags	Tag definitions (slug, label, color, description)
tag_rules	Auto-classification rules: IF field=value THEN apply tag
categories	Taxonomy tree (id, parent_id, name, path)
retention_policies	Retention policy definitions (name, years, action on expiry)
legal_holds	Legal hold records (record_id, applied_by, applied_at, reason, released_at)
saved_searches	Saved search queries per user (user_id, name, query_params JSON)
report_definitions	Custom report builder configs (user_id, name, fields, filters, chart_type)
report_schedules	Scheduled export configs (report_id, cron_expr, format, recipients)
audit_logs	Immutable log — every action on every record
ediscovery_exports	eDiscovery export packages with chain-of-custody log
import_jobs	CSV/XLS bulk import job tracking

5.4 Configurable Field Types (Record Type Schema)

When an Admin creates a custom record type, each field can be one of the following types — matching the client requirement for configurable field schemas:

Field Type	Description	Example Use
text	Single-line text string	Reference number, vendor name
textarea	Multi-line text	Description, notes
number	Integer or decimal	Quantity, shelf number
date	Date picker (YYYY-MM-DD)	Received date, review date
enum	Dropdown — predefined options list	Status, priority, category
boolean	True / False toggle	Confidential, urgent, verified
user	User selector — picks from user list	Assigned to, reviewed by
link	URL or reference to another record	Related record, source document URL

6. Functional Requirements

6.1 Record Capture & Ingestion

Create Record

- Web form renders all 8 required core fields plus any custom fields defined for the selected record type
- Required/optional status and default values are driven by the record type field schema
- Client-side validation (Zod) + server-side validation (Pydantic) — both layers enforced
- Core field rules: Entity Code integer 10–99; Location max 16 chars; Box Barcode exactly 11 chars; File Barcode exactly 13 chars; Year 4-digit integer
- Duplicate check: 409 returned if Box Barcode or File Barcode already exists in DB
- On save: auto-classification rules run → matching tags applied automatically
- On save: version 1 snapshot written to record_versions table
- Audit log entry written in same DB transaction (action = CREATE)

Record Type Templates

- Admin configures record type presets: each preset has a name and a set of custom fields with types, labels, required flag, default values, and validation rules
- Users select a record type at the top of the create form — form fields update dynamically
- Core 8 required fields always present regardless of record type
- Custom JSONB fields stored alongside core fields on the inventory_records row

6.2 Metadata Model & Classification

Tags

- Admin defines tags (slug, label, colour)
- Users can manually apply one or more tags to any record
- Tags are stored as a TEXT[] array on the record and indexed for fast filter queries
- Tag filter available in search and reports

Auto-Classification Rules (Rule-Based — No AI)

- Admin configures rules: IF [field] [operator] [value] THEN apply [tag]
- Supported operators: equals, contains, starts_with, greater_than, less_than
- Rules evaluated server-side on every CREATE and UPDATE
- Multiple rules can apply multiple tags — non-destructive (existing manual tags preserved)
- Example: IF entity_code = 11 AND year < 2020 THEN apply tag 'archival'

Hierarchical Categories (Taxonomy)

- Admin manages a category tree: root categories can have unlimited child levels
- Each record can be assigned to one category
- Category tree browsable in left sidebar on Records page
- Records filterable by category (includes all descendants)

6.3 Search & Retrieval

Full-Text Search

- Global search bar queries entity, department, location, box_barcode, file_barcode, description, year, tags using PostgreSQL tsvector full-text index
- Results ranked by relevance — most relevant first

Advanced Faceted Filters

- Filter by: Entity, Entity Code, Department, Location, Year, Record Type, Tags (multi-select), Category, Retention Status (ACTIVE / DUE / DISPOSED / LEGAL_HOLD), Created By, Date Range (Created At / Updated At)
- Active filters shown as removable chips above the results table
- All filter combinations work together (AND logic)
- Filter state preserved in URL query params — shareable links

Saved Searches

- Any user can save a search+filter combination with a custom name
- Saved searches listed in left sidebar — one click to re-run
- Saved searches are per-user and stored in the saved_searches table
- Users can delete their own saved searches

Barcode Quick-Lookup

- Dedicated input: type any barcode (11 or 13 chars) → instant redirect to that record
- Available from the header nav on every page

Export Results

- Export current search/filter results as CSV — GET /search/export?format=csv
- Export current search/filter results as PDF — GET /search/export?format=pdf (WeasyPrint renders a formatted report)

6.4 Version History, Diff & Revert

- Every UPDATE to a record writes a full snapshot to the record_versions table (version number increments)
- Version history tab on Record Detail page — lists all versions with timestamp and user
- Diff view: side-by-side comparison of any two versions — changed fields highlighted
- Revert: Records Manager or Admin can restore a record to any previous version — revert itself creates a new version (no history is ever deleted)
- Audit log entry written on revert (action = REVERT, from_version, to_version)

6.5 Record Locking — Optimistic Concurrency

- On load, the record form receives the current updated_at timestamp as an ETag

- On submit, the ETag is sent in the If-Match header
- FastAPI checks: if the record's updated_at in DB no longer matches the ETag, return 409 Conflict — 'This record was modified by another user. Please refresh and reapply your changes.'
- This prevents silent data overwrites when two users edit the same record simultaneously
- No pessimistic locking (hard locks) in Phase 1 — optimistic is sufficient for the expected user load

6.6 Retention & Disposition Module

Retention Policies (Admin-Configured)

- Admin creates retention policies: name, retention period (years), action on expiry (flag for review, auto-disposition, notify manager)
- Each record can be assigned a retention policy
- retention_due_date is calculated automatically: Year field + policy years
- APScheduler job runs nightly: updates disposition_status for all records past their due date to DUE

Disposition Actions

- Records Manager can manually mark a record as DISPOSED — writes audit entry (action = DISPOSED)
- Disposition is blocked if legal_hold = TRUE — UI shows clear error message
- Disposed records are hidden from default views but preserved in the DB (soft disposition)

Legal Holds

- Records Manager or Admin can apply a legal hold to any record
- Legal hold blocks all disposition actions — automated and manual
- Legal hold reason is recorded; holds can be released (with reason) by Manager or Admin
- All legal hold apply/release actions are audit logged
- Audit log and eDiscovery exports always include legal hold history

6.7 Audit & Logging

- Every API action is logged: CREATE, READ (optional — configurable), UPDATE, DELETE, REVERT, EXPORT, SEARCH (optional), IMPORT, LEGAL_HOLD_APPLY, LEGAL_HOLD_RELEASE, DISPOSED, LOGIN, LOGOUT
- Each log entry: id, record_id (if applicable), action, performed_by, performed_at, ip_address, changes (JSONB field-level before/after for UPDATE/REVERT)
- Audit table is append-only — no UPDATE or DELETE SQL ever issued against it at application level
- Tamper-evident: each audit row stores a SHA-256 hash of (record_id + action + performed_by + performed_at + changes) — hash chain verifiable by Auditor
- Audit log page: filterable by action type, user, date range, record ID — paginated
- Export audit log as CSV or PDF

6.8 Bulk Import / Export & eDiscovery

Bulk Import

- Accepts CSV (.csv) and Excel (.xlsx) file uploads
- Schema mapping step: user maps upload columns to system fields (drag-and-drop column mapper UI)
- Row-by-row validation via Pydantic — same rules as single-record create
- Error report: table of failed rows with row number, field, and error reason
- User can choose: skip errors and import valid rows, or cancel entire import
- All imported records get audit entries (action = BULK_IMPORT)

Bulk Export

- Export all records (or current filtered set) as CSV or XLSX
- Export includes all core fields + custom fields + tags + category + retention status
- Export action logged in audit (action = EXPORT)

eDiscovery Export

- Records Manager or Admin selects records (by filter or manual selection) and initiates an eDiscovery export
- Export package includes: all record metadata as CSV, full audit trail for each included record, legal hold history, chain-of-custody log (who accessed each record and when)
- Package downloaded as a ZIP file containing CSVs + a cover PDF with export metadata (date, requested by, record count, hash)
- eDiscovery export itself is logged in audit and in the ediscovery_exports table

6.9 Reporting & Analytics

Predefined Reports

- Records by Entity — count + list with bar chart
- Records by Department — count + list
- Records by Year — count + list
- Records by Location — count + list
- Retention Compliance — records by disposition status (ACTIVE / DUE / DISPOSED / LEGAL_HOLD)
- Upcoming Dispositions — records with retention_due_date in the next 30 / 60 / 90 days (configurable)
- Activity by User — records created/updated per user in a date range
- Legal Holds Active — all records currently under legal hold

Custom Report Builder

- Drag-and-drop field selector: choose which fields appear as columns
- Add filters: same faceted filter options as the search page
- Choose chart type: bar, pie, line, or table-only
- Save report configuration with a name — appears in Reports sidebar
- Run saved report at any time with the latest data

Scheduled Exports

- Any saved report can be scheduled for automatic export
- Configure: frequency (daily / weekly / monthly), format (CSV or PDF), recipient email addresses
- APScheduler triggers the job at the configured time — FastAPI-Mail sends the file as an attachment
- Schedule logs stored — Admin can see last run time and status

6.10 Saved Searches

- Search bar and filter panel include a 'Save this search' button
- User names the search — saved to saved_searches table
- Saved searches listed in the left sidebar on the Records page
- One click to restore all search terms and filters exactly
- Users manage their own saved searches; Admins can see all

6.11 External Guest Access

- Admin invites External Guest via email with an expiry date
- Guest receives a magic link to set a password
- Admin configures which records or record subsets the Guest can see (by entity, department, or tag filter)
- Guest sees only a stripped-down read-only view — no audit log, no reports, no admin features
- Guest access automatically expires on the configured date — account is deactivated
- All Guest read actions are logged in audit (action = READ, role = EXTERNAL_GUEST)

6.12 User Management (Admin)

- Create user: email, role, optional expiry date (for Guest accounts)
- System sends welcome email with temporary password via FastAPI-Mail + Gmail SMTP
- Change role, reset password, deactivate account — all from Admin panel
- User list: email, role, status, created date, last login, record count
- Deactivated users cannot log in — their records are fully preserved

6.13 Master Data Management (Admin)

- Manage Entities: add, edit, deactivate — entity_code must be unique
- Manage Departments: add, edit, link to entity
- Manage Locations: add, edit, deactivate
- Manage Record Types & Field Schemas: define custom field types per record type
- Manage Tags: create, edit, assign colours
- Manage Auto-Classification Rules: IF/THEN rule builder UI

- Manage Category Taxonomy: tree editor — add/move/delete nodes
- Manage Retention Policies: name, years, action on expiry

7. API Endpoints (FastAPI) — Base: /api/v1

7.1 Auth

Method	Endpoint	Description	Roles
POST	/auth/login	Login → JWT	All
POST	/auth/logout	Invalidate token	All
GET	/auth/me	Current user + role	All
POST	/auth/change-password	Change own password	All

7.2 Records

Method	Endpoint	Description	Roles
GET	/records	List — paginated, sorted, filtered	All
POST	/records	Create record + auto-classify + version snapshot	Admin, Manager, Worker
GET	/records/{id}	Get record + current version	All
PUT	/records/{id}	Full update with ETag concurrency check	Admin, Manager, Worker*
PATCH	/records/{id}	Partial update with ETag concurrency check	Admin, Manager, Worker*
DELETE	/records/{id}	Soft delete (Admin: ?hard=true for hard delete)	Admin, Manager
GET	/records/{id}/versions	List all versions of a record	All
GET	/records/{id}/versions/{v}	Get a specific version snapshot	All
POST	/records/{id}/revert/{v}	Revert record to version v — creates new version	Admin, Manager
POST	/records/{id}/legal-hold	Apply legal hold with reason	Admin, Manager

Method	Endpoint	Description	Roles
DELETE	/records/{id}/legal-hold	Release legal hold with reason	Admin, Manager
POST	/records/{id}/dispose	Mark record as DISPOSED (blocked if legal hold)	Admin, Manager
GET	/records/barcode/{code}	Look up by Box or File Barcode	All

** Workers can only update records where created_by = their own user ID.*

7.3 Search

Method	Endpoint	Description	Roles
GET	/search	Full-text + faceted filter search, paginated	All
GET	/search/export	Export results — ?format=csv or ?format=pdf	Admin, Manager
GET	/saved-searches	List user's saved searches	All
POST	/saved-searches	Save current search query	All
DELETE	/saved-searches/{id}	Delete a saved search	All

7.4 Import / Export

Method	Endpoint	Description	Roles
GET	/import/template	Download blank CSV template	All
POST	/import/csv	Upload CSV — validate + import	Admin, Manager
POST	/import/xlsx	Upload XLSX — validate + import	Admin, Manager
GET	/import/jobs/{id}	Check import job status + error report	Admin, Manager
GET	/export	Export records — ?format=csv|xlsx|pdf + filters	Admin, Manager
POST	/ediscovery/export	Create eDiscovery export package (ZIP)	Admin, Manager
GET	/ediscovery/exports	List past eDiscovery export packages	Admin, Auditor

7.5 Reports

Method	Endpoint	Description	Roles
GET	/reports/by-entity	Count + list by Entity	Admin, Manager, Auditor
GET	/reports/by-department	Count + list by Department	Admin, Manager, Auditor
GET	/reports/by-year	Count + list by Year	Admin, Manager, Auditor
GET	/reports/by-location	Count + list by Location	Admin, Manager, Auditor
GET	/reports/retention	Retention compliance by status	Admin, Manager, Auditor
GET	/reports/upcoming-dispositions	Records due for disposition soon	Admin, Manager, Auditor
GET	/reports/activity	Records created/updated per user	Admin, Auditor
GET	/reports/legal-holds	All active legal holds	Admin, Manager, Auditor
GET	/reports/custom	Run saved custom report	Admin, Manager, Auditor
POST	/reports/custom	Save a custom report definition	Admin, Manager
GET	/reports/schedules	List scheduled exports	Admin
POST	/reports/schedules	Create a scheduled export	Admin
DELETE	/reports/schedules/{id}	Delete a scheduled export	Admin

All report endpoints accept ?from=YYYY-MM-DD&to=YYYY-MM-DD and ?format=csv|pdf.

7.6 Audit Log

Method	Endpoint	Description	Roles
GET	/audit	List audit log — filter by action, user, date, record	Admin, Auditor
GET	/audit/record/{id}	Full audit history for a specific record	Admin, Auditor, Manager
GET	/audit/export	Download audit log — ?format=csv|pdf	Admin, Auditor
GET	/audit/verify	Verify tamper-evident hash chain integrity	Admin, Auditor

7.7 Classification — Tags, Rules, Taxonomy

Method	Endpoint	Description	Roles
GET	/tags	List all tags	All
POST	/tags	Create tag	Admin
PUT	/tags/{id}	Edit tag	Admin
GET	/tag-rules	List auto-classification rules	Admin
POST	/tag-rules	Create rule	Admin
DELETE	/tag-rules/{id}	Delete rule	Admin
GET	/categories	Get full taxonomy tree	All
POST	/categories	Add category node	Admin
PUT	/categories/{id}	Edit / move category node	Admin
DELETE	/categories/{id}	Delete leaf category node	Admin

7.8 Admin — Users, Master Data, Retention

Method	Endpoint	Description	Roles
GET	/admin/users	List all users	Admin
POST	/admin/users	Create user + send invite	Admin
PUT	/admin/users/{id}	Edit role / expiry / status	Admin
DELETE	/admin/users/{id}	Deactivate user	Admin
GET	/master/entities	List entities	All
POST	/master/entities	Create entity	Admin
GET	/master/departments	List departments	All
POST	/master/departments	Create department	Admin
GET	/master/locations	List locations	All
POST	/master/locations	Create location	Admin
GET	/record-types	List record types + fields	All
POST	/record-types	Create record type	Admin
PUT	/record-types/{id}	Edit record type schema	Admin
GET	/retention-policies	List retention policies	Admin, Manager

Method	Endpoint	Description	Roles
POST	/retention-policies	Create retention policy	Admin
PUT	/retention-policies/{id}	Edit retention policy	Admin

8. Frontend Screens

Screen	Route	Description
Login	/login	Email + password login form
Dashboard	/	Stats: total records, due for disposition, active holds, recent activity feed
Records List	/records	Searchable, filterable, paginated table with faceted filter panel + category tree sidebar + saved searches
Record Detail	/records/:id	All fields, tags, category, retention info, version history tab, audit trail tab, legal hold status
Create Record	/records/new	Dynamic form based on selected record type + all 8 core fields
Edit Record	/records/:id/edit	Pre-filled form with ETag concurrency check
Version Diff	/records/:id/versions	Version list + side-by-side diff view + revert button
Import	/import	Drag-drop CSV/XLSX upload + column mapper + error report table
Reports	/reports	Predefined reports + custom report builder + scheduled exports manager
eDiscovery	/ediscovery	Select records → generate export package → download ZIP
Audit Log	/audit	Full audit log table with filters + hash verify button
Admin — Users	/admin/users	User management: invite, role, expiry, deactivate
Admin — Master Data	/admin/master	Entities, Departments, Locations, Record Types, Retention Policies
Admin — Classification	/admin/classification	Tag manager + auto-classification rule builder + category taxonomy tree editor
Profile	/profile	Current user info + change password

9. Project Folder Structure

9.1 Backend (Python / FastAPI)

```
backend/
  app/
    main.py                          # App init, CORS, router registration
    config.py                        # Settings via pydantic-settings
    database.py                      # SQLAlchemy async engine + session
    dependencies.py                 # Shared deps: get_db, get_current_user,
role_required
    scheduler.py                    # APScheduler setup — retention checks, scheduled
reports

    models/                          # SQLAlchemy ORM models
      record.py                      #   InventoryRecord, RecordVersion
      user.py                        #   User
      audit_log.py                   #   AuditLog
      classification.py              #   Tag, TagRule, Category
      retention.py                   #   RetentionPolicy, LegalHold
      report.py                      #   ReportDefinition, ReportSchedule
      master.py                      #   Entity, Department, Location, RecordType,
RecordTypeField
      import_job.py                  #   ImportJob
      ediscovery.py                  #   EDiscoveryExport

    schemas/                         # Pydantic request/response models
    routers/                         # One file per feature group
      auth.py                        #   /auth/*
      records.py                     #   /records/*  (CRUD + versions + legal hold +
dispose)
      search.py                      #   /search/* + /saved-searches/*
      import_export.py               #   /import/* + /export/* + /ediscovery/*
      reports.py                     #   /reports/* + schedules
      audit.py                       #   /audit/*
      classification.py              #   /tags/* + /tag-rules/* + /categories/*
      admin.py                       #   /admin/users/*
      master.py                      #   /master/* + /record-types/* + /retention-
policies/*

    services/                        # Business logic — keeps routers thin
      record_service.py
      version_service.py             #   Snapshot, diff, revert
      audit_service.py               #   write_audit_log() + hash chain
      classification_service.py      #   Auto-tag rule engine
      retention_service.py           #   Due date calc, disposition check
      import_service.py              #   CSV + XLSX validation + bulk insert
      export_service.py              #   CSV, XLSX, PDF export + eDiscovery ZIP
      report_service.py              #   Report query builder
      email_service.py               #   FastAPI-Mail wrappers

    migrations/                      # Alembic migration files
```

```
      versions/
        001_initial_schema.py
        002_add_versions_tags.py
        003_add_retention_legal_hold.py
        004_add_reports_schedules.py
  requirements.txt
  Dockerfile
  alembic.ini
  .env.example
```

9.2 Frontend (React + Vite + TypeScript)

```
frontend/
  src/
    pages/                          # Route-level page components
    components/
      records/                      #   RecordTable, RecordForm, RecordDetail
      search/                       #   SearchBar, FilterPanel, FacetChips,
SavedSearches
      versions/                     #   VersionList, DiffView
      reports/                      #   ReportCard, CustomReportBuilder,
ScheduleManager
      classification/               #   TagPicker, CategoryTree, RuleBuilder
      audit/                        #   AuditTable, AuditTrail
      import/                       #   DropZone, ColumnMapper, ErrorReport
      ediscovery/                   #   ExportWizard
      ui/                           #   ConfirmDialog, Pagination, Toast, Navbar,
Sidebar
    hooks/                          # useRecords, useSearch, useAuth, useReports,
useAudit
    lib/                            # api.ts, validators.ts, utils.ts, pdfHelper.ts
    types/                          # record.ts, user.ts, auditLog.ts, report.ts
    store/                          # authStore.ts (Zustand)
```

10. Non-Functional Requirements

Category	Requirement
Performance	API responses < 300ms for CRUD; < 2s for search on 500,000+ records; PostgreSQL indexes on box_barcode, file_barcode, entity, department, year, tags, retention_due_date, tsvector column
Security	bcrypt (cost 12) passwords; JWT HS256 with 8hr expiry; all inputs Pydantic-validated; parameterised SQL only; HTTPS enforced; CORS locked to frontend origin
Audit Integrity	audit_logs: append-only at app level; SHA-256 hash chain per row; /audit/verify endpoint confirms integrity

Category	Requirement
Concurrency	Optimistic locking via updated_at ETag on all PUT/PATCH endpoints — 409 returned on stale write
Scalability	FastAPI async + PostgreSQL connection pooling (asyncpg); APScheduler for background jobs; architecture ready for Celery in Phase 2
Availability	99.5% uptime; Supabase managed DB with automated failover; Railway/Render auto-restart
Data Integrity	DB CHECK constraints mirror Pydantic rules; NOT NULL enforced at DB level; foreign key constraints on all FK columns
Backup	Supabase automated daily backups; point-in-time recovery on paid plan
Browser Support	Latest Chrome, Firefox, Edge, Safari — no IE
Responsive Design	Desktop-first (1280px+); usable on tablets (768px+)
Accessibility	WCAG 2.1 AA; shadcn/ui accessible by default; keyboard-navigable forms
Config / Secrets	All secrets in .env — DATABASE_URL, JWT_SECRET_KEY, SMTP creds; never hardcoded or committed
API Docs	FastAPI auto-generates Swagger UI at /docs and ReDoc at /redoc — always current

11. Development Milestones — Phase 1

Week	Phase	Deliverables
Wk 1	Foundation	FastAPI scaffold, SQLAlchemy models (all tables), Alembic migrations, Supabase setup, Dockerfile, JWT auth, role middleware
Wk 2	Core Record CRUD	All record CRUD endpoints + Pydantic validation + version snapshots + optimistic locking + audit log middleware + React form + list page
Wk 3	Search & Classification	Full-text search + faceted filters + barcode lookup + tag engine + auto-classification rules + category taxonomy API + React filter panel + tag picker
Wk 4	Version & Retention	Version history API + diff + revert + retention policy API + disposition + legal hold + APScheduler nightly job + React version diff UI + legal hold UI
Wk 5	Import / Export	CSV + XLSX import with column mapper + error report + CSV/XLSX/PDF export + eDiscovery ZIP export + React import page
Wk 6	Reports	8 predefined reports + custom report builder + scheduled exports (APScheduler + FastAPI-Mail) + recharts in React

Week	Phase	Deliverables
Wk 7	Admin & Master Data	User management + External Guest with expiry + record type schema builder + taxonomy editor + rule builder + all master data endpoints
Wk 8	Dashboard & Polish	Dashboard summary stats, saved searches, responsive layout, loading/error states, empty states, toast notifications
Wk 9	QA & UAT	End-to-end testing, bug fixes, performance profiling, deploy to Vercel + Railway, client UAT, final sign-off

12. Environment Variables

Backend (.env)

```
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/dbname
JWT_SECRET_KEY=long-random-secret-min-32-chars
JWT_ALGORITHM=HS256
JWT_EXPIRY_HOURS=8
ENVIRONMENT=development
ALLOWED_ORIGINS=http://localhost:5173,https://your-app.vercel.app
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your@email.com
SMTP_PASSWORD=app-password
AUDIT_LOG_READ_EVENTS=false   # Set true to log READ actions
```

Frontend (.env)

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
```

13. Open Questions for Client

1. Expected record volume at go-live, 6 months, and 1 year? (Affects index tuning)
2. Are Box Barcode and File Barcode unique globally, or unique within an Entity only?
3. Does the client have a fixed list of Entities and Departments ready for Phase 1, or will they be added as the system is used?
4. Should READ actions be logged in the audit trail? (Configurable — performance trade-off)
5. Who will configure retention policies — System Admin only, or Records Manager too?
6. Should soft-deleted records be visible to Admins in a separate 'Deleted Records' view?
7. Is Gmail SMTP acceptable for notification emails, or is a dedicated email service (e.g. SendGrid free tier) required?

8. For eDiscovery exports — is there a specific format or legal standard that must be followed?
9. Should the External Guest role be able to download records (CSV export) or view only?

14. Phase 2 Preview — AI & Automation (Future)

None of the following is in scope for Phase 1. The FastAPI backend is architected so these features can be added as new service files and endpoints without changing existing code.

Feature	Free Tool	Description
AI field suggestions	Groq API (free — Llama 3)	Suggest entity, department, description based on barcode and history
Natural language search	Google Gemini API (free tier)	'Show all boxes from HR in 2023' → structured query
Auto-classification AI	HuggingFace Inference API (free)	Text classification on description field for smarter tagging
Local private AI	Ollama (self-hosted, 100% free)	Run Llama 3 / Mistral locally — no data leaves the server
Semantic deduplication	sentence-transformers (free, local)	Vector similarity to detect near-duplicate records
Async bulk processing	Celery + Redis	Move CSV import and large exports to background workers
Smart report summaries	Gemini / Groq free tier	AI-generated plain English narrative of report results

15. Glossary

Term	Definition
FastAPI	Python async web framework — REST API with auto Swagger UI
Pydantic	Python data validation — enforces strict types on every API input
SQLAlchemy	Python ORM — maps Python classes to PostgreSQL tables
Alembic	DB migration tool for SQLAlchemy — version-controlled schema changes
APScheduler	Python job scheduler — runs retention checks and scheduled report exports

Term	Definition
WeasyPrint	Python library — renders HTML/CSS to PDF on the server
openpyxl	Python library — reads and writes .xlsx Excel files
JWT	JSON Web Token — signed token with user identity and role for stateless auth
bcrypt	Password hashing algorithm — one-way; passwords never stored in plain text
ETag / Optimistic Locking	Concurrency control — updated_at timestamp used to detect conflicting updates
Zod	TypeScript schema validation in React — mirrors Pydantic rules client-side
Zustand	Lightweight React state management — used for auth state
recharts	React chart library — bar, pie, line charts in reports
Soft Delete	Setting is_active = False — record hidden from views but preserved in DB
Legal Hold	Flag that blocks record disposition — applied for compliance or litigation
Disposition	The action taken when a record's retention period expires (flag, archive, or dispose)
tsvector	PostgreSQL full-text search index type
RLS	Row-Level Security — PostgreSQL feature for per-row access control
eDiscovery	Export of records + chain-of-custody metadata for legal proceedings
Chain of Custody	Log of every user who accessed a record — included in eDiscovery exports
CRUD	Create, Read, Update, Delete — the four basic data operations
PRD	Product Requirements Document — this document

End of Document — Spider Smart Solution IMS | Final PRD v4.0 | React + FastAPI (Python) + Supabase | All Client Requirements Satisfied